

μOS++ III Multi-Core SMP Upstream Integration Plan

Exhaustive Step-by-Step Technical Reference & Architecture Rationale

Baseline: micro-os-plus-iii (xpack-development vs. smp)

Port Ecosystem: posix-arch, cortexm, aarch32, aarch64, devices, riscv

Verification Target: 24 Toolchain/QEMU Builds (72 Unit Tests) 100% Passing at Every Step

1 Architectural Baseline & Ecosystem Topology

1.1 The Ecosystem Landscape

The μOS++ IIIe Real-Time Operating System is structured as a modular, multi-repository architecture where a portable core library communicates with target-specific hardware ports:

Repository	Baseline (xpack-development)	Fork (smp)	Delta & Role
micro-os-plus-iii	Commit 3a22a06b (2026-09-20)	Commit 1a728371	+143 commits, 43 modified files in include/ and src/. Zero new kernel source files added.
micro-os-plus-iii-posix-arch	Tag v1.0.1	smp (+18 commits)	Host-thread-as-CPU model, per-CPU signal masking, compiler barriers.
micro-os-plus-iii-cortexm	Tag v1.1.0	smp (+53 commits)	SysTick ICSR fix, ARMv8-M / Cortex-M33, RP2350 SIO hardware spinlocks.
micro-os-plus-iii-aarch32	Upstream tracking	smp active	RPi3B and RPi Zero 2W support, GICv2 distributor & CPU interface.
micro-os-plus-iii-aarch64	Upstream tracking	smp active	64-bit ARMv8-A execution, GICv2/v3 support, AArch64 MMU translation.
micro-os-plus-iii-devices	Does not exist upstream	smp active	Shared device abstractions, board console mirror, clock definitions.

Core Migration Objective

Progressively transition all bug fixes, architectural enhancements, multi-core SMP infrastructure, and board ports into xpack-development through **30 granular, bisectable, verified steps**, ensuring that all 72 upstream tests pass without modification at every single step.

1.2 The 5 Compilation Defects of Bulk Copying

Directly copying smp onto xpack-development fails compilation across all 24 configurations due to 5 distinct defects:

- Missing Clock Counter API:** src/rtos/os-clocks.cpp calls port::clock_highres::has_hardware_counter(), missing in released ports v1.0.1 / v1.1.0.
- Missing timegm() Prototype:** Removed declaration triggers -Wmissing-prototypes under strict glibc C11/POSIX flags on Linux.
- Alignment & Buffer Diagnostics:** Allocator pointer arithmetic in src/memory/first-fit-top.cpp triggers -Wcast-align and -Wunsafe-buffer-usage.
- Exit-Time Destructors:** Static category objects in src/libcpp/system-error.cpp trigger Clang's -Wexit-time-destructors.
- File Descriptors Pointer Bounds:** Array indexing in src/posix-io/file-descriptors-manager.cpp triggers -Wunsafe-buffer-usage.

2 The Inviolable Migration Rules

- Granular Code Chunks, Not File Copies:** A step consists of minimal, cohesive code snippets (a single function fix, struct definition, or conditional block).

2. **Cross-Repository Synchronization:** When a kernel change requires port support, the kernel and the corresponding ports (posix-arch, cortexm) are updated and linked in the exact same step.
3. **Logical Progression:**
 - **Part A (Steps 1–13):** Uniprocessor bug fixes, memory safety, C++17 conformance, and POSIX I/O hardening (compiled and executed by existing tests).
 - **Part B (Steps 14–23):** Multi-core SMP infrastructure, strictly gated behind `#if defined(OS_USE_SMP_SCHEDULER)` (zero single-core delta verified via `unifdef`).
 - **Part C (Steps 24–28):** Port releases, new architecture files, CMake targets, and add-only test harness extensions.
 - **Part D (Steps 29–30):** Documentation, Typst PDFs, and final merge.
4. **Pristine Test Procedures:** Existing test suites, platforms, and package.json action workflows in `xpack-development` **must never be modified**.
5. **Continuous 72/72 Test Gate:** Every step is accepted only when all 24 compiler/target configurations build cleanly with `-Werror` and pass all 72 test runs.
6. **Add-Only Test Extensions:** New multi-core tests, new platforms, and board scripts are added strictly as new files, new platform directories, and new package.json actions in Part C.

3 Official Verification Gate & Port Linkage Protocol

The official verification gate runs inside `tests/` using `xpm` and `CMake/Ninja`:

```
# Step 1: Register local port working copies for the active step
cd ~/Work/micro-os-plus/micro-os-plus-iii-posix-arch && git switch step/NN && xpm link
cd ~/Work/micro-os-plus/micro-os-plus-iii-cortexm && git switch step/NN && xpm link

# Step 2: Bind local ports across all 24 test configurations
cd ~/Work/micro-os-plus/micro-os-plus-iii/tests
for c in native-cmake-gcc11 native-cmake-gcc12 native-cmake-gcc13 native-cmake-gcc14 \
        native-cmake-clang16 native-cmake-clang17 native-cmake-clang18 native-cmake-clang19 \
        \
        qemu-cortex-m0-cmake-gcc qemu-cortex-m3-cmake-gcc \
        qemu-cortex-m4f-cmake-gcc qemu-cortex-m7f-cmake-gcc; do
    for t in debug release; do
        xpm run link-deps --config ${c}-${t}
    done
done

# Step 3: Run official 72-test gate
xpm run test-all
```

Critical Warning on link-deps-all

Do **NOT** use `xpm run link-deps-all`. Upstream's `tests/package.json` contains a known typo that duplicates `native-cmake-gcc13-debug` and omits `native-cmake-gcc14-debug`, causing GCC 14 builds to silently test stale downloaded packages. Use the explicit loop above.

4 Automated Step Verification Script (`scripts/verify-step.sh`)

The script `scripts/verify-step.sh` automates invariant diff checks, diagnostic linting, port linkage, and test execution:

```
#!/usr/bin/env bash
# scripts/verify-step.sh -- Automated Step Verification Gate
set -euo pipefail

STEP_NUM="${1:-}"
if [ -z "$STEP_NUM" ]; then
    echo "Usage: $0 <step-number (01-30)>" && exit 1
fi
```

```

ROOT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"
WORK_DIR="$(dirname "$ROOT_DIR)"
echo "=== Verifying Step $STEP_NUM in $ROOT_DIR ==="

# 1. Single-Core Unifdef Invariant Verification (for Part B steps 14-23)
if [ "$STEP_NUM" -ge 14 ] && [ "$STEP_NUM" -le 23 ]; then
    echo ">>> [Stage 1] Checking undef -UOS_USE_SMP_SCHEDULER invariant..."
    cd "$ROOT_DIR"
    for f in $(git diff --name-only origin/xpack-development HEAD -- include/ src/); do
        git show origin/xpack-development:"$f" | undef -UOS_USE_SMP_SCHEDULER > /tmp/old_sc ||
true
        git show HEAD:"$f" | undef -UOS_USE_SMP_SCHEDULER > /tmp/
new_sc || true
        if ! diff -u /tmp/old_sc /tmp/new_sc > /tmp/sc_diff; then
            echo "[-] ERROR: Single-core code divergence detected in $f!" && cat /tmp/sc_diff &&
exit 2
        fi
    done
    echo "[+] Unifdef invariant verified: Zero single-core delta."
fi

# 2. Re-link local development ports across all 24 configurations
echo ">>> [Stage 2] Linking local ports..."
cd "$WORK_DIR/micro-os-plus-iii-posix-arch" && xpm link
cd "$WORK_DIR/micro-os-plus-iii-cortexm" && xpm link

cd "$ROOT_DIR/tests"
CONFIGS=(native-cmake-gcc11 native-cmake-gcc12 native-cmake-gcc13 native-cmake-gcc14
        native-cmake-clang16 native-cmake-clang17 native-cmake-clang18 native-cmake-clang19
        qemu-cortex-m0-cmake-gcc qemu-cortex-m3-cmake-gcc
        qemu-cortex-m4f-cmake-gcc qemu-cortex-m7f-cmake-gcc)

for c in "${CONFIGS[@]"; do
    for t in debug release; do xpm run link-deps --config "${c}-${t}" > /dev/null 2>&1; done
done

# 3. Execute the 72-test official suite
echo ">>> [Stage 3] Running official test-all suite (72 tests)..."
xpm run test-all
echo "[SUCCESS] Step $STEP_NUM passed all verification gates!"

```

5 Part A: Uniprocessor Bug Fixes & Code Hardening (Steps 1 – 13)

Part A code changes affect single-core execution and are directly exercised and verified by upstream tests.

5.1 Step 1: ISO C Conformance, Standard Syscalls & Intrusive Lists Iterators

Why:

- posix/dirent.h: Empty struct typedef struct { ; } DIR; violates ISO C99/C11 §6.7.2.1; lone semicolon emits -Wextra-semi.
- libc/stdlib/timegm.c: Strict glibc flags hide timegm(); missing prototype causes -Wmissing-prototypes under -Werror.
- posix-io/c-syscalls-aliases-standard.h: Newlib on AArch64 uses _READ_WRITE_RETURN_TYPE (int), conflicting with unconditional ssize_t.
- utils/lists.h: Iterator operator++(int) and operator-- use node_>next / node_>prev as member variables instead of methods next() / prev().
- rtos/os-thread.cpp: this_thread::suspend() defined inline in .cpp produces no exported symbol, failing C wrapper link.

How:

- Add `int reserved;` to `DIR`. Declare `timegm()` conditionally. Call `node_->next()` / `node_->prev()`. Remove `inline` from `suspend()`.

5.2 Step 2: Dynamic Memory Management, Arithmetic Overflow & Usable Size

Why:

- `align_size(size, align)` wraps on huge size (`size > SIZE_MAX - align + 1`), allocating a tiny block for a huge request.
- `lifo.cpp`: Undersized first chunk pointer not cleared, corrupting allocator metadata.
- `block-pool.cpp`: Inverted assertion `if (res != nullptr) assert(res != nullptr)` instead of `assert(res == nullptr)`.
- `malloc.cpp`: `calloc(nelem, elbytes)` multiplication overflow enables heap buffer overwrite.
- `realloc()`: Copied `new_size` from `old_ptr` without querying old buffer capacity, reading past the old block into unmapped heap memory.

How:

- Wrap checks returning `SIZE_MAX / ENOMEM`. Implement polymorphic `do_usable_size()` in `first_fit_top` and copy `std::min(old_usable_size, new_size)`. Silence `-Wcast-align` and `-Wunsafe-buffer-usage` via `#pragma`.

```
// [src/memory/first-fit-top.cpp]
#pragma GCC diagnostic push
#pragma GCC diagnostic ignored "-Wcast-align"
#ifdef __clang__
#pragma clang diagnostic ignored "-Wunsafe-buffer-usage"
#endif

std::size_t
first_fit_top::do_usable_size (const void* addr) const noexcept
{
    if (addr == nullptr) return 0;
    const chunk_t* chunk = static_cast<const chunk_t*> (addr) - 1;
    return chunk->size - sizeof (chunk_t);
}

#pragma GCC diagnostic pop
```

5.3 Step 3: Modern C++17 Aligned Allocators & Chrono Clock Overflow Protection

Why:

- Missing C++17 aligned operator `new(size, std::align_val_t)` caused runtime fallback to standard `malloc`, bypassing RTOS memory locks.
- `system_error_category()` returned temporary instances, creating dangling pointers in `std::error_code`.
- `chrono::high_resolution_clock` multiplied 64-bit cycles by 1,000,000,000 before division, wrapping in minutes.

How:

- Add 10 aligned `new/delete` operators routing to `os::rtos::memory::alloc/free`. Use function-local static `const system_error_category_impl`. Split `chrono` conversion into integer seconds and remainder.

5.4 Step 4: C API Wrapper Conformance & Polymorphic Destructor Safety

Why:

- `os_timer_create()` defaulted to periodic when `attr == NULL`, diverging from C++ and CMSIS one-shot defaults.
- `os_mutex_delete()` deleted recursive mutexes through base pointer lacking virtual destructor.
- CMSIS v1 32-bit millisecond timeout multiplication wrapped at 71.6 minutes `((uint64_t)(millisec * 1000u))`.

How:

- Default to `timer::once_initializer`. Cast to concrete derived type before `delete`. Widen timeout operand `(uint64_t) millisec * 1000u`.

5.5 Step 5: Thread-Safe POSIX I/O & File Descriptors Manager Protection

Why:

- Concurrently allocating/closing file descriptors caused race conditions and descriptor table corruption.
- Dereferencing empty descriptor slots caused null pointer crashes.
- Inverted `block_device` size check (`size != 0` returned `EINVAL`).

How:

- Wrap allocation/indexing in `interrupts::critical_section`. Validate non-null. Fix check to `size == 0`.

5.6 Step 6: Architecture Port Modernization & ARMv8-M Exception Handling**Why:**

- Cortex-M33 (ARMv8-M) omitted from Thumb processor checks; semihosting issued SVC instead of BKPT.
- Semihosting `fstat()` applied `st_mode |= S_IFCHR` unconditionally, corrupting regular file types.

How:

- Add `__ARM_ARCH_8M_MAIN__ / 8M_BASE` checks and `SecureFault_Handler`. Apply `S_IFCHR` only when mode has no file type. Add `weak os_board_console_mirror()`.

5.7 Step 7: Timer Subsystem Re-entrancy & Tick ISR Safety**Why:**

- Timer callbacks ran inside `interrupts::critical_section` in `SysTick` ISR, blacking out interrupts and deadlocking on synchronization.
- Periodic timer re-arming at `timestamp + period` fired repeatedly if delayed.
- Handler mode `errno` writes crashed asserting `current_thread_`.

How:

- Unlink expired timer under lock, execute callback outside lock, re-arm at `now + period` under lock. Handler mode `__errno()` uses scratch integer.

5.8 Step 8: Mutex Priority Inheritance & Ceiling Protocol Hardening**Why:**

- Priority ceiling checked after acquisition, corrupting owned mutex count on `EINVAL`.
- Priority inheritance lowered boost on arrival of lower-priority waiters.
- Releasing mutex leaked transient boost values into released mutex object.
- Uncritical section window allowed owner to release and nullify `owner_` asynchronously.

How:

- Validate ceiling before ownership. Maintain maximum boost across waiters. Clear `boosted_prio_` on release. Re-verify owner on re-locking.

5.9 Step 9: Thread Lifecycle, Destruction Protocol & State 7 (destroying)**Why:**

- Double-free race between idle reaper and `kill()` during thread termination.
- Lost wakeup in `join()` due to 3-step unsynchronized check-register-sleep sequence.
- Empty `detach()` stub left child threads attached to parents.
- `resume()` on active threads corrupted ready list pointers.

How:

- Introduce `thread::state::destroying` (State 7). Atomic `join()` check and sleep under single lock. Implement `detach()`. Restrict `resume()` to suspended threads.

5.10 Step 10: Condition Variable Atomicity & High-Precision Timeouts**Why:**

- `wait()` unlocked mutex and enqueued non-atomically, losing signals arriving in between.
- `timed_wait()` applied timeout to mutex lock instead of condition signal, and ignored clock attribute.

How:

- Enqueue under scheduler lock, atomically unlock mutex and suspend. Pass clock attribute to timer. Propagate `ETIMEDOUT`. Add condvar trace tag (12).

5.11 Step 11: std::thread Functor Lifetime & Synchronization**Why:**

- `std::thread::join()` deleted native handle before thread completed.
- Functor arguments freed via kernel `func_args_` which was cleared on thread exit, leaking functor objects.

How:

- Call `native_handle()->join()` before deletion. Retain dedicated `function_object_` member in wrapper.

5.12 Step 12: Inter-Thread Message Queue Reschedule Triggers**Why:**

- Waking message queue threads did not trigger immediate preemption on posix-arch, adding up to 1 ms latency until next timer tick.

How:

- Issue `port::scheduler::reschedule()` immediately upon releasing critical section in message queue send/receive.

5.13 Step 13: High-Resolution Hardware Clock Port Synchronization**Why:**

- Kernel invokes `port::clock_highres::has_hardware_counter()` (Defect 1).
- Cortex-M `cycles_since_tick()` checked `SysTick->CTRL` instead of `SCB->ICSR` (bit 26 `PENDSTSET`), causing time-stamps to jump backwards.

How:

- Kernel calls port APIs. Cortex-M returns `false / 0` and checks `SCB->ICSR`. POSIX-arch returns `true` and reads `CLOCK_MONOTONIC`.

6 Part B: Multi-Core SMP Kernel Infrastructure (Steps 14 – 23)

Every snippet in Part B is enclosed within `#if defined(OS_USE_SMP_SCHEDULER)` (with `#else` blocks preserving uniprocessor logic in posix-arch).

- **Step 14: Per-CPU Topology & CPU Identification:** Declare extern `"C" unsigned port_cpu_id(void);` in kernel. In posix-arch, use `thread-local _this_cpu` with `__attribute__((noinline))` and `asm volatile (" ::: "memory")` compiler memory barrier to prevent Clang 16–18 %fs:0 thread pointer caching.
- **Step 15: Per-CPU Scheduler Critical Section:** Replace scalar lock state with `lock_state[OS_NCPU]`. `scheduler::critical_section` sets calling core's lock state without masking hardware interrupts. posix-arch masks `SIGALRM` before querying CPU ID.
- **Step 16: SMP Recursive Kernel Lock:** Introduce struct `smp_klock_t (lock, owner, count)`. Acquire with `std::memory_order_acquire`, release with `std::memory_order_release`. Called inside `interrupts::critical_section`.
- **Step 17: Per-CPU Interrupt State Tracking:** Per-CPU `irq_set` and `_in_isr[OS_NCPU]` in posix-arch. `in_handler_mode()` evaluates `_in_isr[port_cpu_id()]`.
- **Step 18: Per-CPU Active Execution Context:** Transition `current_thread_` to `current_thread_[OS_NCPU]`. `this_thread::thread()` reads pointer with local interrupts masked.
- **Step 19: Per-CPU Idle Thread Instances:** Array `os_idle_thread_core[OS_NCPU]`. Dedicated idle thread registered per CPU at startup.
- **Step 20: Thread CPU Affinity Masking:** `th_cpu_affinity` bitmask attribute and `cpu_affinity()` APIs (default `0xFFFFFFFF`). Main thread pinned to CPU 0 (`1U << 0`).
- **Step 21: 5-Stage Deferred Publish / Claim Context Switch:** Context switch protocol placing `stack_ptr` at offset 0 of context struct. Stage 1: Atomic claim (`to->stack_ptr = nullptr`). Stage 2: Callee register spill. Stage 3: Hardware SP update. Stage 4: Deferred publish (`from->stack_ptr = old_sp`). Stage 5: Register restore.
- **Step 22: SMP Ready List Thread Picker:** Affinity-aware ready list traversal in `internal_switch_threads()`, skipping threads currently running or unpublished (`stack_ptr == nullptr`).
- **Step 23: Host Multiprocessing Emulation (POSIX-Arch):** Spawns `OS_NCPU` host threads, per-CPU `SIGALRM` timers, and IPI signal handling via `pthread_kill(SIGUSR1)`.

7 Part C: Port Releases, New Architectures & Add-Only Tests (Steps 24 – 28)

Part C is strictly **add-only**: existing targets, test runners, and configurations are unmodified.

- **Step 24: Port Releases & Repository Synchronization:** Tag and release `posix-arch v1.1.0`, `cortexm v1.2.0`, `devices v1.0.0`. Merge upstream changes into `aarch32` and `aarch64`.
- **Step 25: New Cores, Boards & Hardware Drivers:** Cortex-M33 (`include-m33/`, `os-core-m33.cpp`) and RP2350 (`include-rp2350/`, `os-core-rp2350.cpp`, SIO Spinlock 0, SIO FIFO IRQ 25). POSIX-arch test harness files (`board-contract.cpp`, `free-store.cpp`).
- **Step 26: Modular CMake Build Architecture:** Add `cmake/toolchains/`, `cmake/uos-app.cmake`, `port/smp-common/`. Introduce granular targets (`micro-os-plus::iii-posix-io`, `micro-os-plus::iii-drivers`) while aliasing `micro-os-plus::iii`.
- **Step 27: New Test Sources & Emulation Linker Scripts:** Add `tests/sources/fp-switch/` (FPU context switch test), `tests/smp-support/` (SMP concurrency stress test), and QEMU MPS2 AN505 / AN521 linker scripts.

- **Step 28: Add-Only Test Platforms & Execution Targets:** Add platform folders under `tests/platforms/` (`2xcortex-m33`, `cortexm-pico2`, `aarch32-rpi3b`, `aarch64-rpi3b`, `native-smp`). Add `test-smp-all` action to `tests/package.json`.

8 Part D: Documentation & Final Upstream Integration (Steps 29 – 30)

- **Step 29: Documentation Suite & Typst PDF Artifacts:** Integrate comprehensive architecture documentation, port guides, comparison papers, and changelogs.
- **Step 30: Final Merge & Upstream Reconciliation:** Re-instate upstream `.github/workflows/ci.yml`, `README.md`, `LICENSE`, and Doxygen trees. Run final 72 uniprocessor + SMP test gate. Open upstream PR.

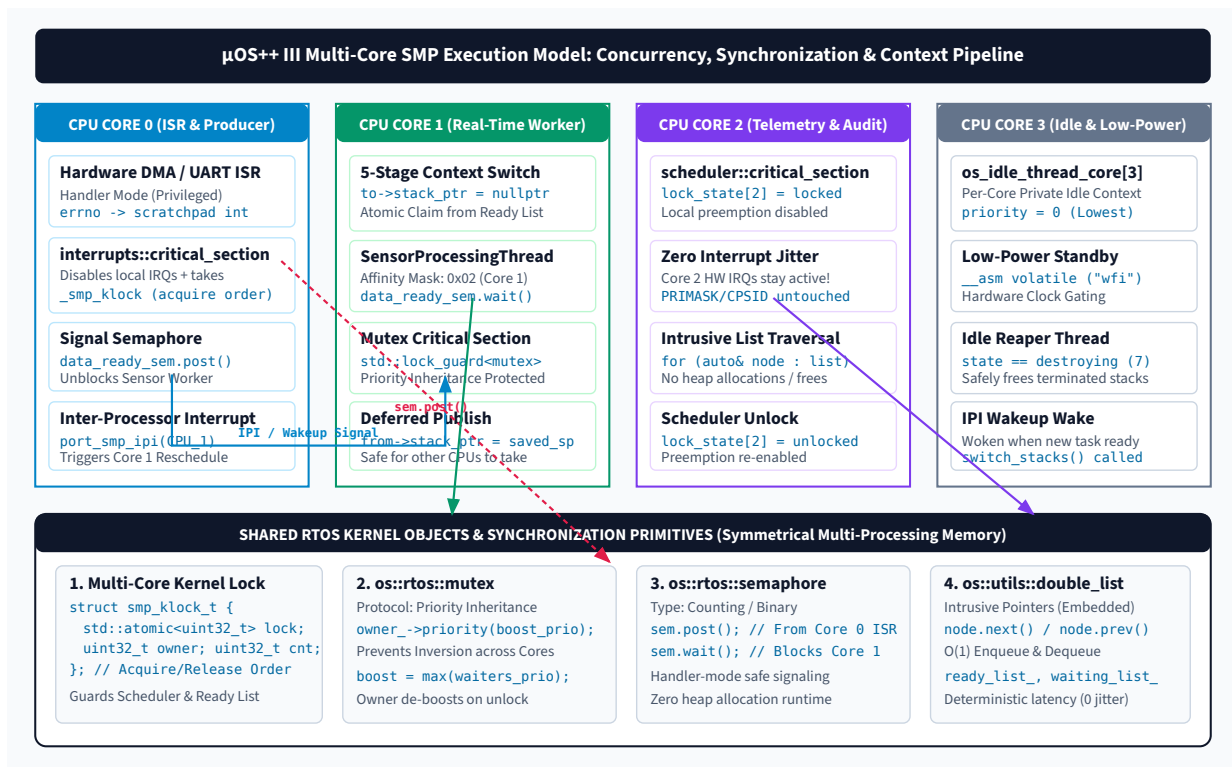
9 Comprehensive Step Execution Checklist

Step	Phase	Core Action / Snippet	Kernel (K)	Cortex-M (C)	POSIX-Arch (P)	Gate Check
01	Part A	ISO C DIR, Glibc <code>timegm()</code> , Newlib types, List iterators	<code>posix/dirent.h</code> , <code>timegm.c</code> , <code>lists.h</code>	—	—	72/72 Pass
02	Part A	<code>align_size</code> wrap check, heap usable size, <code>calloc</code> overflow	<code>os-memory.cpp</code> , <code>first-fit-top.cpp</code>	—	—	72/72 Pass
03	Part A	C++17 <code>operator new(align_val_t)</code> , <code>static system_error</code> , <code>chrono</code>	<code>new.cpp</code> , <code>system-error.cpp</code>	—	—	72/72 Pass
04	Part A	One-shot timer default, polymorphic deletion, 64-bit timeouts	<code>os-c-wrapper.cpp</code>	—	—	72/72 Pass
05	Part A	File descriptor manager <code>mutex</code> ing, block device size fix	<code>file-descriptors-manager.cpp</code>	—	—	72/72 Pass
06	Part A	ARMv8-M mainline macros, <code>semihosting</code> traps, UART mirror	<code>semihosting.h</code> , <code>exception-handlers.c</code>	—	—	72/72 Pass
07	Part A	Timer callback outside critical section, <code>ISR</code> <code>errno</code>	<code>os-lists.cpp</code> , <code>os-timer.cpp</code>	—	—	72/72 Pass
08	Part A	Mutex priority inheritance and ceiling fix	<code>os-mutex.cpp</code>	—	—	72/72 Pass
09	Part A	Thread state destroying (7), <code>atomic join</code> , <code>detach</code>	<code>os-thread.cpp</code> , <code>os-idle.cpp</code>	—	—	72/72 Pass
10	Part A	CondVar <code>atomic list insert</code> + <code>unlock</code> , high-precision timeout	<code>os-condvar.cpp</code>	—	—	72/72 Pass
11	Part A	<code>std::thread::join</code> synchronization, functor lifetime	<code>thread-cpp.h</code>	—	—	72/72 Pass
12	Part A	Message queue <code>reschedule</code> yield triggers	<code>os-mqueue.cpp</code>	—	—	72/72 Pass
13	Part A	<code>clock_highres</code> hardware counter port synchronization	<code>os-clocks.cpp</code>	<code>os-inlines.h</code>	<code>os-inlines.h</code>	72/72 Pass
14	Part B	<code>port_cpu_id()</code> , POSIX-arch compiler barrier	<code>os-core.cpp</code>	<code>os-inlines.h</code>	<code>os-core.cpp</code>	Unifdef + 72/72

15	Part B	Per-CPU scheduler lock state lock_state[OS_NCPU]	—	os-decls.h	os-core.cpp	Unifdef + 72/72
16	Part B	Multi-core recursive kernel lock_smp_klock	os-c-decls.h	os-decls.h	os-core.cpp	Unifdef + 72/72
17	Part B	Per-CPU interrupt tracking (_in_isr[OS_NCPU], irq_set)	—	—	os-decls.h	Unifdef + 72/72
18	Part B	Per-CPU current thread pointer current_thread_[OS_NCPU]	os-sched.h	os-core.cpp	os-core.cpp	Unifdef + 72/72
19	Part B	Per-CPU idle thread instances os_idle_thread_core	os-core.cpp	os-core.cpp	os-core.cpp	Unifdef + 72/72
20	Part B	Thread CPU affinity masking (th_cpu_affinity)	os-thread.cpp	—	—	Unifdef + 72/72
21	Part B	5-stage deferred publish/claim context switch (stack_ptr)	os-idle.cpp	os-core.cpp	os-core.cpp	Unifdef + 72/72
22	Part B	Multi-core ready list thread picker	os-core.cpp	—	—	Unifdef + 72/72
23	Part B	POSIX-arch host thread per CPU, IPI signal engine	os-thread.cpp	—	host_cpu.cpp	Smoke + 72/72
24	Part C	Tag and release ports (posix-arch v1.1.0, cortexm v1.2.0)	—	Release v1.2.0	Release v1.1.0	72/72 Pass
25	Part C	New architecture files (M33, RP2350 spinlocks)	os-thread.cpp	os-core-m33.cpp	board-contract.cpp	72/72 Pass
26	Part C	Modular CMake build scripts and targets	cmake/	—	—	72/72 Pass
27	Part C	New test sources (fp-switch, smp-support, linker scripts)	tests/sources/	—	—	72/72 Pass
28	Part C	Add-only test platforms (2xcortex-m33, pico2, aarch32/64)	tests/platforms/	—	—	72 Old + New Pass
29	Part D	Complete technical documentation & Typst PDFs	docs/	—	—	Clean Doc Build
30	Part D	Final branch alignment, repository restore, upstream PR	All Repos	All Repos	All Repos	Full Pass

10 Holistic Architecture in Action: Complete Multi-Core Example

10.1 System Schematic & Interaction Model



10.2 Complete C++ Application Code

The following listing is a complete, standards-compliant μOS++ III C++ application executing across 4 symmetrical cores:

```
#include <cmsis-plus/rtos/os.h>
#include <cmsis-plus/rtos/lists.h>
#include <array>
#include <stdint>
#include <stdio>

using namespace os::rtos;

// 1. Intrusive Data Structure (Zero Heap Overhead during Runtime)
struct telemetry_sample_t : public os::utils::double_list_links {
    uint32_t timestamp;
    uint32_t sensor_id;
    float    reading;
};
static os::utils::double_list g_telemetry_queue;

// 2. Shared Kernel Synchronization Primitives
static semaphore_counting g_data_ready_sem{ 0, 100 };
static mutex g_shared_buffer_mutex{ mutex::initializer_recursive };

static constexpr size_t POOL_SIZE = 16;
static std::array<telemetry_sample_t, POOL_SIZE> g_sample_pool;
static size_t g_pool_index = 0;

static constexpr size_t RING_BUFFER_SIZE = 32;
static uint32_t g_raw_ring_buffer[RING_BUFFER_SIZE];
static volatile size_t g_ring_head = 0;
```

```
static volatile size_t g_ring_tail = 0;

// 3. Hardware ISR Simulation (Executes on Core 0 in Handler Mode)
extern "C" void dma_uart_hardware_isr (void) {
    // Enter kernel critical section (disables local IRQ, takes _smp_klock with acquire
    // order)
    {
        interrupts::critical_section ics;
        uint32_t raw_data = 0xABCD1234;
        size_t next_head = (g_ring_head + 1) % RING_BUFFER_SIZE;
        if (next_head != g_ring_tail) {
            g_raw_ring_buffer[g_ring_head] = raw_data;
            g_ring_head = next_head;
        }
    } // _smp_klock released with release order, local IRQ restored

    // Post semaphore to unblock consumer thread on Core 1
    g_data_ready_sem.post ();
}

// 4. Real-Time Consumer Thread (Pinned to CPU Core 1)
static void* sensor_processing_thread_func (void* args) {
    (void)args;
    while (true) {
        result_t res = g_data_ready_sem.wait (); // Atomically sleeps, 5-stage switch
        if (res != result::ok) continue;

        uint32_t extracted_data = 0;
        // Mutex with Priority Inheritance: boosts lower-priority owner if contended
        {
            std::lock_guard<mutex> lock (g_shared_buffer_mutex);
            if (g_ring_tail != g_ring_head) {
                extracted_data = g_raw_ring_buffer[g_ring_tail];
                g_ring_tail = (g_ring_tail + 1) % RING_BUFFER_SIZE;
            }
        }

        // Enqueue onto intrusive list under critical section
        {
            interrupts::critical_section ics;
            telemetry_sample_t* sample = &g_sample_pool[g_pool_index++ % POOL_SIZE];
            sample->timestamp = static_cast<uint32_t> (clock_systick::now ());
            sample->sensor_id = 1;
            sample->reading = static_cast<float> (extracted_data & 0xFFFF) * 0.01f;
            g_telemetry_queue.link_tail (*sample); // 0(1) intrusive enqueue, zero heap
allocation
        }
    }
    return nullptr;
}

// 5. Telemetry & Audit Thread (Pinned to CPU Core 2)
static void* telemetry_worker_thread_func (void* args) {
    (void)args;
    while (true) {
        clock_systick::sleep_for (100);

        // Scheduler Critical Section: lock_state[Core2] = locked.
        // Local preemption disabled, BUT Core 2 hardware interrupts remain active (Zero
        Jitter)!
```

```

{
    scheduler::critical_section scs;
    size_t count = 0;
    for (auto& node : g_telemetry_queue) {
        telemetry_sample_t& sample = static_cast<telemetry_sample_t&> (node);
        count++;
    }
    trace::printf ("[Core %u] Telemetry sweep processed %zu items.\n", port_cpu_id (),
count);
    } // Preemption re-enabled on Core 2
    }
    return nullptr;
}

// 6. Application Initialization & Core Startup (Main Thread on Core 0)
int os_main (int argc, char* argv[]) {
    thread::attributes sensor_attr = thread::initializer;
    sensor_attr.th_priority = thread::priority::high;
    #if defined(OS_USE_SMP_SCHEDULER)
        sensor_attr.th_cpu_affinity = (1U << 1); // Pinned to Core 1
    #endif
    thread sensor_thread{ "sensor-consumer", sensor_processing_thread_func, nullptr,
sensor_attr };

    thread::attributes telemetry_attr = thread::initializer;
    telemetry_attr.th_priority = thread::priority::normal;
    #if defined(OS_USE_SMP_SCHEDULER)
        telemetry_attr.th_cpu_affinity = (1U << 2); // Pinned to Core 2
    #endif
    thread telemetry_thread{ "telemetry-audit", telemetry_worker_thread_func, nullptr,
telemetry_attr };

    for (size_t i = 0; i < 5; ++i) {
        clock_systick::sleep_for (50);
        dma_uart_hardware_isr ();
    }
    return 0;
}

```

10.3 Chronological State Transition Trace

Time	Core	Execution Context	Action / Primitive	Memory & Hardware State
T0	C0	os_main()	Launches Threads, sets Affinity	C1 aff=0x02, C2 aff=0x04
T1	C1	SensorWorker	g_data_ready_sem.wait()	C1 enters 5-stage switch to Idle
T2	C2	TelemetryWorker	sleep_for(100)	C2 switches to Idle Core 2
T3	C3	IdleCore[3]	__asm volatile("wfi")	Core 3 enters low-power standby
T4	C0	dma_uart_isr	Enters interrupts::crit_sec	Local CPSID + _smp_klock acquired
T5	C0	dma_uart_isr	g_data_ready_sem.post()	SensorWorker moved to ready list
T6	C0	dma_uart_isr	port_smp_ipi(CPU_1)	Hardware IPI signal sent to Core 1
T7	C1	IPI Handler	Reschedule triggered	Core 1 executes switch_stacks()
T8	C1	Stage 1 Switch	to->stack_ptr = nullptr	Atomic claim: locked against C2/C3
T9	C1	Stage 4 Switch	from->stack_ptr saved_sp =	Deferred publish: Idle published
T10	C1	SensorWorker	lock_guard<mutex>	Mutex acquired; priority unchanged

T11	C2	TelemetryWorker	scheduler::crit_sec	lock_state[2]=1, Core 2 IRQ OPEN
T12	C2	TelemetryWorker	double_list iteration	Traverses node.next() without heap
T13	C2	TelemetryWorker	Exits scheduler::crit_sec	lock_state[2]=0, preemption active

— End of Technical Document —